One-Month Project Plan: **Local RAG** AI Assistant with Microsoft **Foundry Local**

Man link: https://techcommunity.microsoft.com/blog/azuredevcommunityblog/building-your-first-local-rag-application-with-foundry-local/4501968

Executive summary

Phase 1: Foundations

Learn RAG basics and set up Foundry Local on Windows & macOS. Cover embeddings, vector search, SQLite, and prompt design.

Phase 2: Build Project

1–2 weeks of coding a local Q&A assistant. Implement document ingestion, retrieval pipeline, local LLM integration via Foundry Local.

Phase 3: Testing & Wrap-up

1–2 weeks for final project testing (functional & evaluation) and writing comprehensive documentation. Final demo/presentation.

The goal is to **guide beginner computer science students through a *full-time, one-month summer program in which they build a local Q&A/knowledge assistant using Microsoft Foundry Local for offline model inference and the RAG (Retrieval-Augmented Generation) pattern. The final project is inspired by the Microsoft Tech Community example of a local RAG support agent that answers questions from documents with zero Internet dependency. By the program's end, each student team will have a working offline Q&A chatbot that can answer questions about a small document collection (e.g. course notes, manuals, FAQs) by retrieving information locally and integrating it into a large language model's responses.*** *[azurefeeds.com]*

Program Structure: We divide the month into three phases, each focusing on specific skills and deliverables:

- **Phase 1 – Foundational Learning (Weeks 1–2):** Introduction to RAG concepts, Foundry Local, embeddings, vector search, SQLite, and prompt engineering fundamentals.
- **Phase 2 – Project Implementation (Weeks 3–4):** Hands-on development of the RAG application, from data ingestion and retrieval pipeline to app integration with the on-device LLM (Large Language Model).
- **Phase 3 – Testing & Documentation (Week 5, and optionally Week 6):** System testing and evaluation, performance tuning, and preparing project documentation and final presentations.

Below, each phase is broken down by week with learning objectives, resources, **hands-on exercises**, and **milestones**. The schedule is adaptable (some topics may overlap between weeks as needed for scaffolding). **Official Microsoft resources and high-quality tutorials** are provided for each topic. (Community blog sources are noted as such.)

Project Overview & Key Technologies

Before diving into the weekly plan, let's clarify **the project and its components**:

- **Project Aim:** Build a **local document Q&A assistant** – a simple chatbot that runs entirely on a student's computer. It uses **Retrieval-Augmented Generation (RAG)** to answer user questions by *retrieving relevant content from a local knowledge base of documents and feeding it to a local LLM* for answer generation. This results in **fewer hallucinations and more accurate, source-grounded answers**. The assistant can run offline (no internet needed) via **Microsoft Foundry Local**, which provides an *on-device LLM runtime*. [azurefeeds.com]
- **Foundry Local:** *What is it?* Foundry Local is an **end-to-end local AI solution** that provides a lightweight runtime and SDK for running large language models completely *on a user's device*, with a curated catalog of optimized models. **No cloud account or GPU is required** – Foundry Local automatically downloads & manages models and runs inference with CPU/NPU acceleration, so apps can deliver **local, offline AI with zero network calls**. This is key to our project, as it lets students experiment with an **LLM (e.g. a smaller GPT variant) locally**. [learn.microsoft.com] [azurefeeds.com]
- **Retrieval-Augmented Generation (RAG):** RAG is an **AI design pattern** where you **Retrieve** relevant information from a document set, **Augment** the model's input prompt with that info as context, then have the model **Generate** an answer. The model's responses are thus *grounded in your own data* (reducing hallucination and enabling source citations) by combining **embedding-based semantic search** with an

LLM. We'll explore *why* RAG is useful (especially for custom Q&A agents) and compare it with simpler context injection methods. [azurefeeds.com] [learn.microsoft.com]

- **Embeddings & Vector Search:** We'll introduce **text embeddings** (numerical vector representations of text meaning) and how to use them for **semantic similarity search**. Students will learn how RAG typically relies on an *embedding model* to convert documents into vectors for storing in a *vector database*, and how queries can be embedded and matched with relevant documents by measuring vector similarity. We will see this both conceptually and in practice with Foundry Local's embedding capabilities. [learn.microsoft.com]
- **SQLite for Local Data:** We use **SQLite** as a lightweight local database to store document texts and their embeddings. SQLite is a *serverless, self-contained SQL database* (just a single file) and is the world's most widely deployed database engine. Its advantages include *no separate server, cross-platform support, and simple integration*, making it ideal for local data storage. We'll cover basic SQL/SQLite usage so students can confidently use it to manage their documents and vectors (e.g. storing and retrieving *embedding vectors* and text chunks). [sqlite.org] [learn.microsoft.com]
- **Prompt Engineering:** Crafting good LLM prompts (especially **system prompts** for role instructions and **user prompts** for queries) is vital for effective answers. We'll discuss basic **prompt engineering techniques** for Q&A tasks, such as instructing the model to cite sources and not answer if unsure. Students will practice writing simple prompts and understand how prompt design can influence the model's behavior.
- **Project Architecture:** The final application will have a straightforward architecture with all components on one machine. It consists of a **client interface** (e.g. a basic web UI or console input for questions), a **server/pipeline** layer that handles user queries and orchestrates retrieval & generation, a **data layer** (SQLite database storing document embeddings), and an **AI layer** (the Foundry Local LLM performing on-device inference). Below is a visual overview of this architecture: [azurefeeds.com]

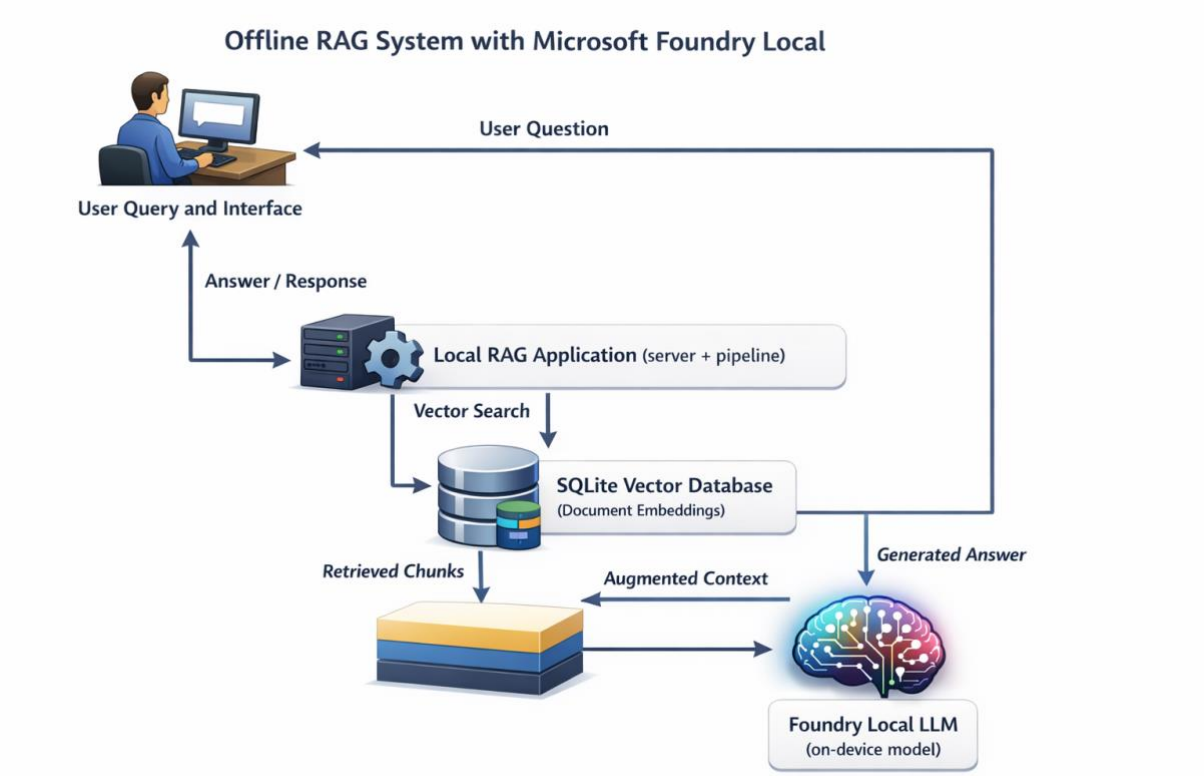


Figure 1: Architecture of a local RAG (Retrieval-Augmented Generation) system using Foundry Local on a single device. The user's query is processed by a local application which retrieves relevant document chunks from a SQLite vector database ("vector search"), then sends the query plus retrieved context ("augmented context") to a local LLM via the Foundry Local SDK. The LLM's answer is returned to the user – all with no internet connection needed. [azurefeeds.com]

Learning Map: Key Components & Resources

Foundry Local (On-Device LLM Runtime): Official docs explain Foundry Local's purpose and features.

RAG Fundamentals & Use Case:

Tech Community blog (Beginner-friendly) provides a scenario-based intro to RAG and its benefits.

Embeddings & Vector Search:

Microsoft Learn tutorial shows how to generate embeddings & query them with Foundry Local.

SQLite Basics:

Learn about SQLite's usage for lightweight local storage and practice simple SQL queries.

Prompt Engineering 101:

Foundry docs on prompt writing best practices & roles for guiding model outputs.

[learn.microsoft.com], [azurefeeds.com], [learn.microsoft.com], [learn.microsoft.com], [learn.microsoft.com]

Above are the **core topics and references** that will guide instruction and self-study. With the context set, we can now detail the **phase-by-phase plan**:

Phase 1 (Weeks 1–2): Foundational Learning

Objectives: Build a strong conceptual foundation in RAG and local AI tools. By end of Week 2, students should **understand how RAG works** and be comfortable with the basic technologies and development environment. They'll have **Foundry Local installed and tested on their laptops (both Windows & macOS)**, a sample **SQLite database** created, and perhaps small test programs run (embedding generation and vector similarity search).

Week 1: *RAG Concept & Local AI Setup*

Topics & Activities:

- **Intro to RAG (Retrieval-Augmented Generation):** Start by explaining the *problem* that RAG solves. Use simple examples: ask a general LLM about a specific domain question (likely gets it wrong), then explain how RAG can incorporate external knowledge to improve answers. Cover RAG's **"retrieve, augment, generate"** steps and the resulting benefits (more accurate answers, reduced hallucinations). [azurefeeds.com]
    - *Resource: Microsoft Tech Community* blog post *"Building Your First Local RAG Application with Foundry Local"* – **read the introduction and "What is RAG" section** for an accessible overview (community content). [azurefeeds.com]
    - *Exercise:* **Q&A role-play:** Provide a short document or knowledge base (1 page) and have students simulate RAG manually – one acts as the "retriever" (finding the relevant paragraph), another as the "LLM" (formulating an answer using that info). This illustrates how adding context helps answer questions.
- **Understanding Foundry Local & Environment Setup:** Introduce **Microsoft Foundry Local** and why it's key to our project. Emphasize that it allows running an **LLM entirely offline on student laptops** (no cloud needed), and mention supported platforms (Windows/macOS/Linux). Cover Foundry Local's main features: on-device model downloads, hardware acceleration (utilizing CPU/GPU/NPU automatically), and an easy-to-use SDK for Python (and other languages). [azurefeeds.com] [learn.microsoft.com]
    - *Resource:* **Official documentation – "What is Foundry Local?"** for a high-level overview. Also see **Microsoft Learn's "Get started with Foundry Local" guide** (select *Python* tab) for step-by-step installation instructions. [learn.microsoft.com] [learn.microsoft.com], [learn.microsoft.com]

    - *Exercise:* **Install Foundry Local SDK** (latest version) on each student's machine (ensuring both Windows and macOS installations are tested). Follow the official instructions to install via pip (`pip install foundry-local-sdk` or OS-specific variant). To verify the setup, run a *"Hello Model" test:* e.g., writing a short Python script to use Foundry Local's SDK to load a small model (like `phi-1.5-mini`) and generate a simple completion (e.g., given prompt "Hello, world" have it complete the greeting). This confirms the runtime is working and introduces the API. [learn.microsoft.com]

- **Basic Python App Structure:** (If needed due to student background) Review how to structure a Python project: using a `main.py` with a clear entry point (`if __name__ == "__main__": main()`), dividing code into functions or modules (for clarity as the project grows), and managing dependencies via `requirements.txt`. Use a simple example (like a "Hello LLM" project) to illustrate how to organize code.
    - *Resource:* **Microsoft Learn –** *"Tutorial: Build a RAG application".* Start reading the section *"Prerequisites"* and the part about setting up `main.py`. The sample code will serve as a template for our project structure. [learn.microsoft.com]
    - *Exercise:* **Create a skeleton project:** Initialize a new Python project folder for the RAG assistant. Create a `main.py` file and test it by printing a greeting. If using a code editor like VS Code, ensure all students can run the program in their environment.

Milestones by end of Week 1: All students have **Foundry Local installed and working** on their machines, have a basic project folder with a `main.py` file, and can run a trivial Foundry Local inference (e.g., output from a local model to confirm proper installation).

Week 2: *Core Techniques – Embeddings, Vector Search & SQLite*

Topics & Activities:

- **Embeddings & Vector Similarity:** Introduce the concept of **text embeddings** – numeric vector representations of text that capture semantic meaning. Explain how similar text → similar vectors, enabling *semantic search*. This is essential for RAG's retrieval step. Discuss how one can get embeddings (via specialized models like *OpenAI's ada* or local ones) and measure similarity (e.g., via **cosine similarity**). [learn.microsoft.com]
    - *Resource:* **Microsoft Learn – "Tutorial: Build a RAG application"**, sections *"Generate document embeddings"* and *"Search for relevant documents"* (official). This shows how to use Foundry Local's Python SDK to generate embeddings for a list of documents and perform a similarity search by computing cosine similarity of vectors. [learn.microsoft.com], [learn.microsoft.com]

    - *Exercise:* **Embedding demo:** Provide a small list of example sentences and have students use Foundry Local's SDK to generate embeddings for each sentence (using a small embedding model like `qwen3-embedding-0.6b`). Then, for a given query (also embedded), have them code a simple loop to compute similarity scores and find the top match. This can be based on the official tutorial's code (which already includes a `find_relevant()` function) and will solidify how vector search works. [learn.microsoft.com], [learn.microsoft.com]

- **Storing & Querying Embeddings with SQLite:** Explain why we might need a **vector store** or database when scaling beyond a few in-memory documents. Introduce **SQLite** as a *fast, serverless local database engine* perfect for our use-case (no separate server or setup; a single file holds all data). Ensure students understand basic **SQL** operations (creating tables, inserting and querying data) at least conceptually. [learn.microsoft.com]
    - *Resource: "Benefits of SQLite for local storage"* (section in *Microsoft Windows App Development* documentation) for key points on SQLite's advantages. Optionally, use a beginner-friendly **SQLite tutorial** (e.g., **W3Schools SQL tutorial**, "SQLite SELECT" pages – third-party, clearly marked as such). [learn.microsoft.com]
    - *Exercise:* **SQL sandbox:** Have students install the sqlite3 command-line tool or use a simple Python script with the built-in `sqlite3` module to create a small database (e.g., a `documents` table with `id`, `content`, `embedding` fields). Let them practice **inserting** a few sample rows and running a **query** to retrieve a record by an `id` or filter by a text keyword. This gives them familiarity with the process of persisting and retrieving data – skills needed when they integrate SQLite into the RAG pipeline.
- **Basic Prompt Engineering for Q&A:** Discuss how simply retrieving documents isn't enough – *how we present the info to the model matters*. Introduce the concept of **system vs user prompts** (roles in the Chat Completion API) and how to instruct the model to use the retrieved text and not guess beyond it. Share simple guidelines like *"If you don't find info in context, say you don't know"* or *"always include source names in the answer"*.
    - *Resource:* **Microsoft Learn – "Prompt engineering techniques"**, especially the *Basics* of prompt construction and use of *system messages* (official). [learn.microsoft.com]
    - *Exercise:* **Prompt experiments:** Using a public web AI (like Bing Chat or ChatGPT, which they've likely encountered), have students try providing the *same question with and without additional context* (e.g., "Answer using only

this info: [provide a passage]"). Observe how context changes the answer. This is a conceptual parallel to what their RAG chatbot will do.

Milestones by end of Week 2: Students have a working knowledge of **RAG, Foundry Local, embeddings, and SQLite.** They have created a test SQLite database (or at least conceptually designed the schema for storing documents and vectors). They have also practiced retrieving similar text via computing cosine similarity on embeddings in Python, and they understand how to phrase basic prompts for the model. **All prerequisites for hands-on building are in place.** [learn.microsoft.com], [learn.microsoft.com]

Phase 2 (Weeks 3–4): Project Implementation

Objectives: Over ~2 weeks, students will **develop a functional local RAG application**. They will apply what they learned to implement each component: document ingestion, vectorization, retrieval, and LLM integration. By the end of Phase 2, each student (or team) will have a **working offline Q&A chatbot** that can answer questions by retrieving info from its local document store before invoking the model.

Week 3: *Data Ingestion & Retrieval Pipeline*

Topics & Activities:

- **Designing the Knowledge Base & Data Prep:** Decide on the small set of documents for the Q&A assistant (e.g. 5–10 short documents such as technical articles, product FAQs, or course notes – provided by the instructor or chosen by students). Discuss strategies for splitting documents into chunks (as RAG typically works at passage-level chunks, e.g., ~1–3 paragraphs each). Students will implement a **data ingestion script** to: [azurefeeds.com]

1. **Chunk** their documents into smaller passages.
2. Compute an **embedding** for each chunk using Foundry Local's embedding model.
3. **Store** each chunk and its embedding vector in SQLite for later retrieval.

- *Resource:* **Microsoft Learn – "Build a RAG application"** (official). Follow the sections on *creating a knowledge base* and *generating embeddings*. The tutorial's sample code (Python) can be adapted: it shows reading documents, generating embeddings in batch, and saving them in a Python list (we'll extend this to saving in SQLite). [learn.microsoft.com], [learn.microsoft.com]

- *Exercise:* **Code the Ingestion Pipeline:** In their `main.py` (or a separate script/module), each student writes code to open each document, split it (e.g., by paragraphs or headings), and use Foundry Local's SDK to embed each chunk. Use the Python `sqlite3` library to insert each chunk (text and embedding vector) into the database.

For simplicity, the embedding can be stored as a blob or text (JSON-serialized vector). **Test**: After running ingestion, verify the DB has the expected number of entries. Optionally, build a simple **setup script** to re-run ingestion if documents are added or changed.

- **Building the Retrieval Function:** With the data in place, implement the logic to retrieve relevant chunks when given a new user query. Students will:

1. **Embed the query** (using the same embedding model).
2. **Search for similar vectors** in SQLite – here simplest approach is to fetch all stored embeddings (or use a SQL extension if available) and compute cosine similarity in Python, then pick top-K chunks. *(For small N, this is fine; discuss that for large N, specialized vector DBs or using SQL extensions would be needed)*. [learn.microsoft.com], [learn.microsoft.com]
3. **Return** the top chunks as context.

- *Resource:* **Microsoft's tutorial code** for `find_relevant()` (it brute-forces similarity for each document, suitable for our scale). Also reference the Tech Community blog section on **RAG pipeline** decisions (how many chunks to retrieve, etc.). [learn.microsoft.com], [learn.microsoft.com]
- *Exercise:* **Implement & Test Query Retrieval:** In code, implement a function `get_top_chunks(query)` that returns, say, 2–3 most relevant chunks from the SQLite DB given a user query. Test this function with a few sample queries against the current data. For example, ask questions that you know are covered by one of the documents and verify the retrieved text chunks seem relevant. If using SQLite, this might involve reading all vectors into memory to compare with the query's vector (acceptable for our small dataset). If comfortable, an additional challenge: use an **SQL query** with a custom distance function or rely on *vector similarity* features if using an external vector DB – but this is optional.

Milestones by end of Week 3: Students have a **populated SQLite document database with embeddings**, and a working **retrieval function** that can find the top relevant document chunks for a given query.

Week 4: *LLM Integration & Application Assembly*

Topics & Activities:

- **Integrating the Local LLM (Foundry Local Chat Model):** Now connect this retrieval mechanism to a language model to generate answers. Students will load a suitable **small LLM** via Foundry Local (e.g., *Phi-3.5 Mini or similar 3–5B parameter model*) and use it to produce **chat-style** answers. Discuss model selection trade-offs: smaller

models respond faster, but larger ones provide better answers – in this program we prioritize speed so students get quick feedback.

  - *Resource:* **Foundry Local Quickstart** section on using the *native chat completions API* (for multi-turn chat). However, a simpler path: treat it like one-turn Q&A. Ensure students see how to create a **chat client** from the model and call `completeChat` or similar as shown in documentation. [azurefeeds.com], [azurefeeds.com]

  - *Exercise:* **Model Warm-up:** Students write code to load their chosen Foundry Local model at program startup (if not already loaded during ingestion). They then write a function `answer_query(user_question)` that uses `get_top_chunks()` from Week 3 to retrieve context, and then calls the local model's chat API with a **system message** instructing the model to answer using the context (and not outside knowledge) followed by the **user's question**. **Test the pipeline end-to-end** with a straightforward question and inspect the answer. [azurefeeds.com]

- **Building a Simple User Interface:** Depending on time and student skill, provide options for how the Q&A assistant's interface will work:

  - **Option A: CLI** – simplest route. Students can prompt for an input query via the console, call `answer_query(query)`, and print the answer. This keeps focus on backend logic.

  - **Option B: Streamlit or Gradio UI** (Python) – great for visualization. Provide a minimal Streamlit app that calls `answer_query` when the user submits a question. This gives a simple web interface (and is cross-platform).

  - **Option C: Basic HTML+JS UI** – (if students want a web dev experience). They could reuse the blog's approach: a static HTML page with a text box and some JavaScript that calls a local Flask or Node server endpoint for answers. Choosing Option A ensures completion within timeframe; Options B/C can be offered to advanced students or as stretch goals for Week 5 if time remains.

  - *Resource:* **Streamlit documentation** (third-party) or a *basic Flask/Express tutorial* (if Option C). Optionally, show how the Tech Community blog's sample was a browser-based UI served by an Express.js backend. [azurefeeds.com]

  - *Exercise:* **Finalize the App Interface:** Implement the chosen UI. For CLI, loop `input()` to let user ask multiple questions. For a web UI, follow the tutorial to set up minimal UI elements and integrate with the backend. *Test thoroughly:* ask various questions through the interface to verify end-to-end functionality

– ensure the retrieval is happening (maybe log retrieved chunks for verification) and the model responds coherently.

- **Ensuring Responsible Outputs:** Remind students to apply prompt engineering best practices: e.g., always include the instruction that if context is insufficient the assistant should say it doesn't know **(to avoid fabricating an answer)**. Encourage them to fine-tune their system prompt for polite, concise answers. If time permits, have them implement *source citations*: e.g., by storing a short source name with each chunk and having the model give an answer with reference ("according to Document X ..."). This can be done by adding instructions in the prompt or simple post-processing if needed.

Milestones by end of Week 4: Each team has a **working Q&A application** that can take a user's question (through their chosen interface) and return an answer generated by the local LLM, using retrieved content from their *SQLite-backed knowledge base*. The core project functionality is complete.

Phase 3 (Week 5–6): Testing, Evaluation & Documentation

Objectives: In the final phase, students will refine and test their applications, evaluate performance and accuracy, and produce documentation and a presentation. By the end of this phase, projects should be polished and ready to demonstrate.

Week 5: *System Testing & Evaluation*

Topics & Activities:

- **Functional Testing:** Students develop test cases to ensure their assistant works for a variety of queries (both queries it can answer and ones it should *not* be able to). They should verify that:
    - The system **returns an answer with relevant info** when the answer is in the documents.
    - It **responds appropriately when information is missing** (e.g., returns a fallback message like "I don't have that information" per the system prompt instruction).
    - It handles edge cases (like empty query input, or very general questions). *Approach:* Students can compile a small set of Q\&As (some answerable from the docs, some unanswerable) to systematically test their bots. If possible, swap test questions between teams to simulate "real users." They should run their program for each test query and capture the outputs.

- **Performance & Debugging:** Since everything is local, check that response times are reasonable (for small models, e.g., ~1–3 seconds per question on a typical laptop). If responses are slow, discuss potential optimizations (e.g., retrieving fewer chunks, using a smaller model, or ensuring not to recompute embeddings repeatedly by caching them). Help students debug any outstanding issues like incorrect retrieval results or formatting problems in answers.
- **Evaluation and Improvement:** Encourage self-critique and iteration. Have students consider questions like: *Are answers accurate? Are they well-written and concise? Are sources cited (if that was a goal)?* If not, how can they refine their approach (e.g., adjust the prompt format, retrieve more context, or improve chunk splitting)? This is an opportunity for them to apply critical thinking and improve their project's quality.

Milestone by mid Week 5: Test results documented – a list of queries attempted and whether the responses were correct/appropriate. Students should identify any shortcomings and plan final adjustments.

Week 6 (or end of Week 5): *Documentation & Final Presentation*

(Depending on the available time, documentation and presentation prep can overlap with testing in late Week 5, or extend into Week 6.)

Topics & Activities:

- **Project Documentation:** Each team writes a short **Project Report / README** detailing: the project's purpose, how it works, instructions to run the app, and any design decisions or limitations. Ensure they include how to set up the environment and use the assistant (this also reinforces their understanding). If Microsoft Learn modules or tutorials were followed, students should reference those.
- **Code Cleanup & Comments:** Students review their code for clarity. They should remove debug prints, add comments explaining key sections (e.g., what the retrieval function is doing), and ensure their code style is consistent. This is a good point to briefly mention version control (if not already used) to highlight best practices, though a thorough Git/GitHub intro is likely beyond scope unless already known.
- **Final Presentation Prep:** Each group will give a **short demo and presentation** of their local RAG assistant at the end of the program. Guide them to highlight:
    - **Problem Statement:** What scenario or user need does their assistant target?
    - **Key Features/Components:** Briefly explain how it uses RAG, what data sources they included, etc.

- **Live Demo:** Show the assistant answering a few sample questions, including one where it cites a source or says it doesn't know (to prove it handles uncertain queries responsibly).
- **Lessons Learned:** One or two insights or challenges they overcame (e.g., "we learned that splitting documents properly was crucial for good retrieval results"). Optionally, encourage creative elements – like naming their assistant or customizing the interface – to build presentation confidence and engagement.

Milestones by end of Week 6: All teams have **completed projects with documentation** (a final report or README) and have rehearsed their **presentations/demos**. The program concludes with a **demo day** where each team presents their working local RAG assistant, reflecting on what they learned.

Throughout the program, emphasis is on **hands-on learning** reinforced by curated resources. By following Microsoft's official guidance (installation instructions, tutorials) combined with practical coding exercises, students gradually build confidence and competence in each component before integrating them into the final project. **By the end of the month, they will have a functional, offline AI Q&A system** – and a solid understanding of how modern AI applications can be built by combining **retrieval (search) and generation (LLM)** capabilities and running them locally.

This plan ensures that the core topics – Foundry Local, RAG, embeddings, vector search, data management, and prompt engineering – are introduced early and then applied in a stepwise fashion, giving beginners the knowledge and experience to successfully implement and showcase their first local RAG application.